

Unprotected Internal Zap Swap in ZendexAmmManager.addLiquidity

Sandwich attack against single-sided LP deposits caused by a zero-slippage internal swap

73.72 WZEN attacker profit	~24.6% profit on 300 WZEN	200 WZEN victim deposit
--------------------------------------	-------------------------------------	-----------------------------------

Target: HackenProof Zendex / zendex-sc-main

Finding: Valid Medium finding; ultimately closed as out of scope under the program's Critical-only reward rules, with no reputation penalty.

Report created: 20 August 2026

Portfolio case study — independent security research

1. Executive Summary

I identified a sandwichable internal swap in `ZendexAmmManager.addLiquidity()`. The function accepts a single asset from a shielded deposit and internally converts part of it into the paired asset before minting an LP position. The conversion is executed through the public router with `amountOutMin: 0`.

The caller's supplied slippage value does not protect this first swap. Instead, the later LP minimums are calculated from the already-manipulated output of the internal swap. An attacker can therefore move the AMM price before the victim's transaction, allow the victim's zero-slippage zap to execute at the manipulated price, and then reverse the position.

A full Foundry proof of concept reproduced the attack against the real, unmodified Zendex contracts used in the tested path. The attacker started with 300 WZEN and recovered 373.72 WZEN after the front-run, victim execution, and back-run — a measured 73.72 WZEN (~24.6%) increase.

The submitted report was reviewed by HackenProof triage and confirmed as a genuine remediation gap. Triage recorded it as a valid Medium finding, noting that the remaining defect was one leg of the add-liquidity operation and that the outer bound was anchored to the output of the already-unprotected swap. The program closed it as out of scope because only Critical issues were eligible for reward.

2. Finding at a Glance

Component	<code>ZendexAmmManager.addLiquidity</code> → <code>_executeAddLiquidity</code> → <code>_performOptimalSwap</code>
Root cause	Internal router swap uses <code>amountOutMin: 0</code>
Attack primitive	Permissionless AMM price manipulation + victim's forced internal swap
Victim action	Single-sided LP deposit; no attacker cooperation required
Measured result	300 WZEN → 373.72 WZEN; +73.72 WZEN (~24.6%)
PoC	Foundry 1.7.1; 1 test; compiled and passing
Final triage	Valid Medium; closed out of scope under Critical-only reward rules

3. Technical Root Cause

The vulnerable path is the internal zap performed during single-sided liquidity addition. The caller supplies a slippage value in the proof, capped by the protocol at 5%, but that value is not used to protect the internal swap itself.

Vulnerable execution sequence

- `addLiquidity()` receives a shielded deposit and the caller's proven action parameters.
- `_executeAddLiquidity()` invokes `_performOptimalSwap()` when reserves exist.
- `_performOptimalSwap()` calls `router.swapExactTokensForTokens()`.
- The internal swap specifies `amountOutMin: 0`.
- Only after that swap does the later LP operation apply the caller's slippage factor to quantities that already depend on the manipulated swap output.

The critical invariant is therefore broken: the value received by the forced conversion is not bounded against a pre-manipulation reference price.

```
uint256[] memory amounts = $.router.swapExactTokensForTokens( IRouter.SwapExactTokensForTokensParams({
    amountIn: amountToSwap, amountOutMin: 0, path: path, to: address(this), deadline: block.timestamp + 300 }) );
```

The submitted report also explains why the later 5% bound cannot repair this: `amountBDesired` is the output of the already-executed internal swap, so the minimum moves with the degraded output instead of constraining the first leg.

4. Attack Path

- Victim has a valid shielded deposit and an add-liquidity proof.
- Attacker observes the pending add-liquidity transaction and front-runs with a large swap in the same direction as the victim's internal zap.
- The victim's `addLiquidity()` executes against the manipulated reserves.
- The internal zap accepts the manipulated price because `amountOutMin == 0`.
- The victim's own forced swap adds further price impact.
- Attacker back-runs in the opposite direction and captures the resulting round-trip value.

The attack requires no privileged role and no victim cooperation beyond the victim submitting the ordinary add-liquidity transaction.

5. Proof of Concept

The supplied PoC states that it compiles and passes against the real, unmodified zendex-sc-main contracts. It deploys the real protocol components used by the path, including ZendexAmmManager, ZendexRouter, ZendexPair, ZendexFactory, ZendexVault, ZendexVaultManager, TreeOperator, WZEN, RewardsEngine, BoostManager, ZendexStaking, and ZendexVerifierHub.

The only substitution is a MockVerifier for the DEPOSIT, INCLUSION, and ADD_LIQUIDITY proof types. The PoC explains that genuine Noir/Barretenberg proof generation requires a JavaScript toolchain unavailable inside Foundry. The substitution bypasses only cryptographic proof verification; the AMM and liquidity-management business logic under test remains unmodified.

Observed test result

Forge 1.7.1	Tests 1 passed	Gas 3,130,609
-----------------------	--------------------------	-------------------------

```
attacker WZEN profit 73721873717922144786 victim deposit amount 20000000000000000000 wzen recovered in  
back-run 373721873717922144786 Suite result: 1 passed; 0 failed; 0 skipped
```

Attack sequence reproduced by the test

- Seed a WZEN/USDT pool with 500 WZEN and 500,000 USDT.
- Victim deposits 200 WZEN through the real vault-manager/tree path.
- Attacker sells 300 WZEN for USDT.
- Victim executes the real ammManager.addLiquidity() path.
- Attacker uses the USDT from the front-run to buy WZEN back.
- PoC asserts the attacker's WZEN balance increased.

6. Validation & Evidence

The PoC specifically records that the victim's deposit root and epoch are derived from a genuine `TreeOperator` insertion caused by a real `vaultManager.deposit()` call. The attacker trades use the unmodified router and pair, including the protocol's real fee split. The victim's add-liquidity call reaches the shipped `_performOptimalSwap → router.swapExactTokensForTokens(amountOutMin: 0)` path.

This matters for portfolio purposes because the demonstration is not a toy calculation of a constant-product AMM. The supplied test wires the actual contracts around the vulnerable path and uses a real on-chain state transition for the victim's deposit.

7. Impact Analysis

- **Direct economic impact:** the attacker can extract value from a victim's single-sided LP deposit by manipulating the pool around the victim's forced conversion.
- **Permissionless:** the attacker does not require an admin or protocol role.
- **Victim interaction:** the victim does not need to sign an attacker-controlled action; the ordinary add-liquidity transaction supplies the required execution.
- **Atomicity:** the attack is a front-run / victim execution / back-run sequence.
- **Observed magnitude:** 73.72 WZEN profit from a 300 WZEN front-run in the supplied PoC.

The observed percentage is a PoC result for the tested pool state and trade sizes; it should not be interpreted as a universal extraction percentage for every pool or deposit.

8. Remediation Consideration

The supplied report recommends passing the caller's already-proven slippage bound into `_performOptimalSwap()` and using it to calculate a real `amountOutMin` for the internal zap leg, for example by obtaining a router quote and applying the same discount factor used later.

The key design requirement is that the minimum received amount must be anchored to a reference that exists before the attacker can manipulate the pool, rather than to the output of the already-executed swap.

9. Research Methodology

This finding illustrates a protocol-level review approach focused on economic invariants and execution assumptions rather than isolated code patterns.

- **Trace value flow:** follow the deposited asset into the internal swap and LP minting path.
- **Check caller-controlled bounds:** identify where slippage supplied by the caller is actually enforced.
- **Separate pre-trade and post-trade references:** test whether a minimum is anchored to an independent quote or to an already-manipulated value.
- **Model adversarial ordering:** consider front-run and back-run transactions around externally visible state-changing operations.
- **Validate economically:** reproduce the path with a full Foundry test and measure the attacker's net asset balance.

The result is a useful example of how a seemingly present slippage parameter can fail to provide economic protection when it is applied at the wrong stage of a multi-step operation.

10. Triage Outcome

HackenProof triage confirmed the finding as a genuine remediation gap and recorded it as a valid Medium finding. The triage note states that the previous fix covered the pool-deposit leg, while the internal swap remained at `amountOutMin: 0` and the outer bound remained anchored to the output of that swap.

The report was closed as out of scope because the program's rules made only Critical findings eligible for reward. The triage response explicitly preserved the Medium severity assessment and stated that the submission carried no reputation penalty.

11. Portfolio Notes

This case study is presented as independent security research and vulnerability disclosure, not as a commissioned client audit. It demonstrates hands-on Solidity/DeFi security research, economic attack modeling, protocol integration analysis, and Foundry-based validation.

Primary skills: Smart Contract Security · Solidity · DeFi · Economic Invariant Analysis

Tools: Foundry · Solidity · GitHub · AMM/DeFi analysis

Target: HackenProof Zendex / zendex-sc-main

Report date: 20 August 2026

PoC: Foundry test `ZapSandwichTest.test_zeroSlippageZapIsSandwiched()`

Source note: technical claims and PoC figures in this case study are based on the submitted vulnerability report and the supplied Foundry PoC. The PoC source identifies the exact test result, affected execution path, scope substitution, and measured attacker profit.